AI-Facilitated IaC Security Review: The Effects of Model Selection, Context Engineering, and Agent Orchestration on Vulnerability Detection

Author: Clifton Sigler, clifton@collusion.org

Advisor: Dr. Tim Proffitt

Accepted: TBD

Abstract

Infrastructure-as-code written with LLM coding tools is outpacing the human review it depends on. This study measures whether an LLM can take on part of that review and which implementation choices matter: the model, the prompt, the agent pattern, and native reasoning. Four models (Claude Opus 5, Claude Sonnet 5, GPT-5.6 Terra, and a DeepSeek Flash endpoint) reviewed six files from a mutated TerraGoat corpus, rewritten to reduce recall from the training data, across five prompt variants and four orchestration patterns, for 1,080 scored runs in all. Model choice moved F1 from 0.459 to 0.688, almost entirely through recall: the cheaper models were not wrong more often, they found less. Prompt design and multi-agent patterns helped only the two weakest models, by five to eight F1 points, lowered the two strongest, and cost up to 3.6 times as much. Native reasoning helped no model. An audit of the rejected claims in the model comparison found that a third were real weaknesses that the Checkov-derived ground truth could not detect, and two-fifths were true observations outside the study's criterion; adjusted precision ranged from 0.77 to 0.96. Teams should choose a model for recall, keep the prompt plain when the model is strong, and allocate an orchestration budget to a better model. The harness, corpus, and results are published for reproduction.

1. Introduction

Organizations using Large Language Model (LLM)-enabled AI coding tools as part of their Infrastructure-as-Code (IaC) development practices report increases in developer productivity and reductions in time to deploy cloud resources (Crosley, 2026). Unfortunately, as IaC coding velocity ramps up, so too does the volume of IaC that may contain cloud resource misconfigurations. Defining infrastructure as code does not by itself prevent misconfiguration (Verdet et al., 2025), and a misconfigured object store, database, or security group can expose sensitive systems and data, causing significant financial and reputational harm.

The OWASP Top 10 for LLM Applications 2025 (Open Worldwide Application Security Project [OWASP], 2024) specifically highlights the potential for unsafe code generation as part of LLM09:2025 Misinformation. The recommendations for addressing and remediating security vulnerabilities in LLM-generated IaC remain largely the same as for human-generated IaC, with a heavy emphasis on static analysis tools and human review. While human review is often a compliance or regulatory requirement, it continues to limit and undermine the speed and productivity gains that LLMs provide.

While frontier AI labs continue to improve the capabilities of their LLMs, the next important area where IaC developers and cloud engineers can increase productivity and improve cloud security is leveraging LLMs to review IaC for vulnerabilities, thereby reducing reliance on resource-intensive human-in-the-loop review. Recent papers on the subject have demonstrated that LLMs can perform IaC security reviews when properly configured (Firouzi et al., 2026). Performance and capability may be influenced by the implementation decisions developers must make, such as which model to select, how to structure the input context, and whether to use a single-agent or multi-agent architecture. These design choices may affect detection accuracy, however existing research provides insufficient guidance on these implementation trade-offs, particularly regarding IaC security reviews.

To help IaC developers and cloud security engineers more effectively identify vulnerabilities in LLM-generated IaC, this research examines whether model selection,

context engineering, and AI agent orchestration patterns influence the efficacy of AI-facilitated security code reviews for cloud IaC compared with default configurations.

1.1. Thesis

IaC development speed is no longer the bottleneck for cloud engineering teams. Security review is. This paper examines how LLMs can best be used to close that gap. Four models are evaluated against TerraGoat, Bridgecrew's deliberately vulnerable Terraform repository, across multiple context engineering techniques and orchestration patterns, with each run scored on precision, recall, and F1. These insights will enable cloud engineering teams and security organizations to adopt AI-driven IaC security review workflows that scale alongside the productivity gains LLMs already provide.

2. Research Method

This study takes a quantitative, experimental approach. Four large language models review deliberately vulnerable Terraform configurations, and their reported findings are scored against a validated ground truth. Three factors vary across runs: the model performing the review, the context engineering technique shaping its input, and the orchestration pattern coordinating the review process. Native reasoning is disabled in all runs. The model comparison was repeated with reasoning enabled to check whether it changed the ranking. Detection performance is measured by precision, recall, and F1 score, each reported with 95% bootstrap confidence intervals. In all, 36 configurations and 1,080 model runs were scored. The harness, the mutated corpus, every raw model response, and the scripts that score them and regenerate every figure in this paper are published in the study's repository.

2.1. Test Corpus and Ground Truth

The test corpus is TerraGoat, a vulnerable-by-design Terraform repository published by Bridgecrew, now part of Palo Alto Networks (Bridgecrew, 2020b). It was chosen over LLM-generated IaC because its flaws are deliberate and can be enumerated and checked; no public corpus of LLM-generated IaC with a validated ground truth was available. Its

AWS configurations cover the resource categories that the CIS AWS Foundations Benchmark addresses: object storage, compute and network rules, identity policies, databases, and encryption keys (Center for Internet Security, 2026). Six of the AWS files have been reviewed: s3.tf, ec2.tf, db-app.tf, iam.tf, kms.tf, and lambda.tf, 40 resources in all. A seventh, rds.tf, was considered and excluded because it repeats the same database cluster nine times and would have contributed 82 near-duplicate findings; the six files cover the categories above, and the remaining AWS files were not reviewed.

TerraGoat does not provide a machine-readable list of its flaws, so ground truth was derived from a Checkov scan (Bridgecrew, 2020a). Checkov 3.3.11 reports 95 failed checks across the six files. Each check was then classified against a single criterion: a finding is a security vulnerability if it directly creates or widens an attack path. Recovery controls such as versioning and backups are excluded; defense-in-depth controls are included; detective controls are counted only where they produce an alert that a security team can act on, so passive logging is excluded. Checks that were operational rather than security were excluded. Missing public access blocks on S3 buckets, security groups open to 0.0.0.0/0, unencrypted storage, IMDSv1 left enabled, hard-coded secrets in user data, and IAM policies that permit privilege escalation are in; lifecycle rules, cross-region replication, Multi-AZ, deletion protection, monitoring, and security-group descriptions are out. The 52 remaining findings form the ground truth. Each records the affected resource, issue category, CWE mapping, severity, and originating Checkov rule. Several are separate checks on a single attribute or policy document, so recall is measured against controls rather than against distinct flaws.

As most LLMs have been trained on significant amounts of public code from GitHub, the experiments were run on a mutated version of TerraGoat: every resource name and reference, identifier string, HCL comment, and provenance tag was changed or removed, and block and attribute order was shuffled, while the vulnerable attributes themselves were left untouched (Section 3.4.1 lists what the rewrite left behind). The mutated code was then scanned with Checkov 3.3.11 and produced the same 95 failed checks as the original, on the corresponding resources.

2.2. Models Under Test

Four models are selected, spanning a spectrum of capability, cost, and deployment. Table 1 summarizes the selection.

Table 1: Models Under Test

Model	Tier	Selection Rationale
Claude Opus 5	Frontier, high capability	Within-family scaling comparison (Anthropic, 2026)
Claude Sonnet 5	Frontier, mid-tier	Baseline: the tier most teams deploy (Anthropic, 2026)
GPT-5.6 Terra	Frontier, mid-tier	Cross-vendor comparison (OpenAI, 2026)
DeepSeek Flash	Open weights, hosted API	Near-frontier capability claims at a fraction of frontier cost (DeepSeek, 2026)

The Claude Opus 5 and Sonnet 5 pair tests whether added capability within one model family improves detection. GPT-5.6 Terra checks whether mid-tier findings generalize across vendors. For the DeepSeek arm, DeepSeek V4 Flash (deepseek-v4-flash) was used, an open-weights model available through a low-cost hosted API and released under the MIT license (DeepSeek, 2026). Open weights allow for self-hosting, which matters to organizations whose policies prohibit sending proprietary code to external services. The model identifier each API returned is recorded with every run.

2.3. Experimental Variables

Five context variants are tested on all four models. Zero-shot prompting supplies only the code and a review instruction. Few-shot prompting adds hand-crafted examples of vulnerable and corrected Terraform code, none of which are drawn from the test corpus (Brown et al., 2020). System-prompt enrichment establishes a security-reviewer persona carrying CWE-referenced domain knowledge. Guidance injection prepends excerpts from the CIS AWS Foundations Benchmark (Center for Internet Security, 2026), the AWS Well-Architected Framework Security Pillar (Amazon Web Services, n.d.), and the OWASP Infrastructure as Code Security Cheat Sheet (OWASP, n.d.), a static form of retrieval-augmented generation (Lewis et al., 2020). Chain-of-thought prompting directs the model to reason step by step before reporting (Wei et al., 2022). System-prompt

enrichment, the strongest variant in a pilot on the original corpus, is the fixed prompt for the model comparison and the orchestration sweep.

Orchestration is tested through four patterns on all four models: a single agent reviewing the full file, a chunked agent that splits input by resource block and merges deduplicated findings, a critic pattern in which a second agent challenges the first reviewer's output, and a debate pattern in which two independent reviews are reconciled by a third agent (Du et al., 2024).

2.4. Scoring and Metrics

Every model answers through a fixed JSON schema that names the resource and gives a description, a severity, and a CWE class for each claimed finding, so scoring is automatic. A claim matches a ground-truth finding in one of two ways. The first is by resource and category: the claim names the same resource, and its category or description contains at least one cue from the category's vocabulary, a word or phrase such as public or 0.0.0.0/0. The second is by CWE class on the same resource. Where more than one pairing is possible, the pairing with more cues wins, a category match beats a CWE match, and each claim and each finding can be used once. A claim that covers two findings is therefore credited once and the other finding counts as missed, and a second claim on a finding already credited is a false positive. Each claim is therefore a true positive or a false positive, each unmatched finding is a false negative, and the counts give precision (TP / [TP + FP]), recall (TP / [TP + FN]), and F1, their harmonic mean. Latency, token use, and API errors are recorded as secondary metrics for the cost and reliability analysis.

Repetition is required because sampling varies. LLM inference is not deterministic even at temperature zero (He, 2025). Every configuration therefore ran five times per file, which is what the budget and time allowed. All metrics carry 95% percentile bootstrap confidence intervals. Precision, recall, and F1 come from counts pooled over a configuration's 30 runs. The confidence intervals resample those 30 runs 10,000 times. The paired comparisons resample the 30 file-and-repetition units that two configurations

share 20,000 times, and the two-sided p is twice the share of resampled differences that land on the far side of zero.

The ground truth is the list of checks Checkov failed, so it contains only what Checkov can see. When a model reports a real vulnerability that Checkov has no rule for, the scorer counts it as a false positive, and the model's precision comes out lower than it deserves. Precision as scored is therefore a floor. Recall is not affected, because it counts only the 52 planted findings. To measure how far below its true value the floor sits, all 177 claims the scorer rejected in the model comparison were read against the Terraform and sorted into four groups: real vulnerabilities the ground truth lacks, true observations the study's criterion excludes, model errors, and duplicates of findings already credited. Adjusted precision counts the first group as correct and appears beside the raw figure. Recall cannot be adjusted the same way, because the ground truth was not extended, so no adjusted F1 is reported.

2.5. Circularity Avoidance

Ground truth descends from Checkov, so two safeguards prevent the evaluation from scoring the oracle against itself. Guidance text draws only on the independent sources named above, never on Checkov rule definitions. Few-shot examples come entirely from outside TerraGoat.

2.6. Threats to Validity

Several limitations bound these results. TerraGoat is a single synthetic corpus, and its deliberate flaws may not mirror the subtlety of production misconfigurations, so findings should be read as relative comparisons rather than absolute detection rates. The ground truth comes from Checkov and was not extended, so a model that reports a real weakness that Checkov has no rule for is scored as wrong; precision is a floor, and models that report the most are penalized the most. The adjusted precision in Section 3.1 estimates the size of this effect for the model comparison.

Every prompt variant and every orchestration pattern ran on all four models, but the patterns ran on one prompt only, and reasoning was compared only in the model

comparison. Five repetitions per file yield confidence intervals about ±0.05 wide, so small effects are only detected in the paired comparisons. Thirty-five paired tests were run without correction for multiple comparisons, so one or two results at $p < 0.05$ could be chance; the conclusions rest on differences at $p \leq 0.01$. Open-weight benchmark claims remain vendor-reported and unverified (DeepSeek, 2026). Costs are token counts at August 2026 list prices, not billed amounts.

3. Findings and Discussion

Every configuration in this section reviewed the same six mutated TerraGoat files five times each, 30 runs per configuration, scored against the same 52 findings. The model comparison in Section 3.1 holds the prompt and the agent pattern fixed and varies the model. The context leg in Section 3.2 runs all five prompt variants on all four models. The orchestration leg in Section 3.3 runs all four patterns on all four models with the enriched prompt. Including the reasoning check, the study covers 36 configurations and 1,080 model runs: 20 model-and-prompt configurations, 12 model-and-pattern configurations beyond the single call, and 4 reasoning configurations, with the enriched single call serving as the baseline for both the orchestration and reasoning legs. A run is one review of one file; a single-call run makes one API call, a critic run two, a debate run three, and a chunked run one per chunk (1.8 on average), 1,540 calls in all. Because 52 findings are scored five times per configuration, true positives and false negatives sum to 260 in every row of Table 2. No run ended in an API error; the one run that returned no content, a DeepSeek chain-of-thought run on lambda.tf, is scored as empty. Every match came from the resource-plus-category or CWE level (Section 2.4). Section 3.4 reports two checks on the measurement itself: the corpus mutation and the audit of rejected claims.

3.1. Model Selection Testing and Findings

3.1.1. Introduction

This leg asks which model finds the most planted vulnerabilities when everything else is held fixed. Each model reviewed each file with the system-prompt-enriched context and a

single call. Two predictions were recorded before the runs. A general-purpose leaderboard, the Artificial Analysis Intelligence Index, ranked the four models GPT-5.6 Terra, Claude Opus 5, Claude Sonnet 5, and DeepSeek V4 Flash in September 2026, placing GPT first (Artificial Analysis, 2026). SWE-bench Verified, a coding benchmark, ranked Opus above Sonnet, above DeepSeek, and did not list GPT-5.6 Terra (BenchLM, 2026). Two secondary questions were recorded with them: whether the model that reports the most is also the one that is wrong the most, and whether enabling native reasoning changes the ranking.

3.1.2. Presentation of Data

Table 2 reports precision, recall, F1, and the 95% confidence interval for each model, along with the pooled true positive, false positive, and false negative counts and the API cost across its 30 runs. Claude Opus 5 led with an F1 of 0.688, followed by Claude Sonnet 5 at 0.632, DeepSeek Flash at 0.577, and GPT-5.6 Terra at 0.459. Table 3 pairs each model against Opus on the shared file-and-repetition units; all three differences are negative, and all three intervals exclude zero. Table 4 breaks F1 out by file.

Table 2: Detection Performance by Model (System-Prompt Enrichment, Single Agent; 30 Runs Each)

Model	Precision	Recall	F1	F1 95% CI	TP	FP	FN	Cost (USD)
Claude Opus 5	0.684	0.692	0.688	[0.636, 0.735]	180	83	80	1.41
Claude Sonnet 5	0.835	0.508	0.632	[0.576, 0.682]	132	26	128	0.43
DeepSeek Flash	0.696	0.492	0.577	[0.517, 0.633]	128	56	132	0.01
GPT-5.6 Terra	0.871	0.312	0.459	[0.405, 0.512]	81	12	179	0.18

Table 3: Paired F1 Difference Against Claude Opus 5

Model	F1 Difference	95% CI	p
Claude Sonnet 5	−0.057	[−0.082, −0.034]	<0.001
DeepSeek Flash	−0.112	[−0.152, −0.071]	<0.001
GPT-5.6 Terra	−0.229	[−0.277, −0.185]	<0.001

Table 4: Per-File F1 by Model (System-Prompt Enrichment, Single Agent)

Model	db-app.tf (13)	ec2.tf (15)	iam.tf (7)	kms.tf (2)	lambda.tf (5)	s3.tf (10)
Claude Opus 5	0.60	0.73	0.34	1.00	0.87	0.76
Claude Sonnet 5	0.56	0.69	0.33	0.89	0.75	0.72
DeepSeek Flash	0.50	0.63	0.27	0.75	0.58	0.73
GPT-5.6 Terra	0.44	0.42	0.22	0.17	0.57	0.67

Note. Ground-truth findings per file in parentheses.

3.1.3. Detailed Analysis

The gap between models is recall, not precision. Precision ran from 0.68 to 0.87, and the weakest model was the most precise. Recall ran from 0.69 down to 0.31. Over 30 runs, GPT-5.6 Terra reported 93 findings to Opus's 263, and 81 of GPT's were correct. The cheaper models were not wrong more often. They found less. For a security review, that is the costlier failure: a false positive costs a reviewer a few minutes, and a false negative ships a misconfiguration. The order does not depend on the matcher's weakest rule: discounting every single-cue and CWE-level match lowers the four F1 values to 0.570, 0.536, 0.459, and 0.283, while leaving the order unchanged.

Within one family, Opus beat Sonnet by 0.057 F1 (Table 3), and the difference is entirely recall, 0.692 against 0.508. Sonnet was the more precise model, 0.835 against 0.684. At list prices, Sonnet's 30 runs cost $0.43 to Opus's $1.41, so Sonnet delivered 92% of Opus's F1 for 31% of the cost.

Across vendors, the general leaderboard predicted the wrong order. GPT-5.6 Terra, ranked above Opus on the Intelligence Index, finished last on this task, 0.229 F1 below Opus. DeepSeek Flash, ranked last of the four, finished third, within 0.055 of Sonnet at about one thirtieth of Sonnet's cost. The coding benchmark's order, Opus above Sonnet above DeepSeek, held (Table 7). A model's standing on a general index did not predict its performance on this task; its standing on a code-specific benchmark did, for the three models the benchmark listed.

Table 4 shows that file difficulty is the largest source of variation within a model. The iam.tf file, six of whose seven findings are separate checks on a single IAM policy document rather than on single attributes, was the hardest file for the three stronger models, with F1 between 0.27 and 0.34, and Opus's advantage there was the smallest.

The kms.tf file, with two findings, was the easiest for the Claude models and the hardest for GPT, which returned an empty findings list on it in three of five runs; with two findings, one miss halves recall. These swings are why the paired comparisons in Tables 3, 9, and 11 are the study's tests: pairing on file and repetition removes the file effect that the per-configuration intervals include.

The audit of rejected claims described in Section 3.4 changes the precision column without changing the order. Of the 177 claims the scorer rejected in this leg, 64 were real weaknesses the Checkov-derived ground truth had no rule for, 74 were true observations the study's criterion excludes, 34 duplicated a finding the same run had already been credited for, and 5 were wrong. Crediting the first group raises precision to 0.795 for Opus, 0.918 for Sonnet, 0.957 for GPT, and 0.772 for DeepSeek (Table 5); the ranking is unchanged. These are bounds rather than corrected scores: they rest on a single rater. The weaknesses the ground truth missed were not obscure: the database password interpolated into EC2 user data, the IAM access key written in plaintext to Terraform state, MySQL ingress open to the whole VPC rather than the application's security group, snapshots of unencrypted volumes, and buckets with no TLS-only policy. All four models reported the first two.

Table 5: Classification of Rejected Claims and Adjusted Precision (Model Comparison)

Model	Real, outside ground truth	Excluded by criterion	Model error	Duplicate	Rejected total	Raw precision	Adjusted precision
Claude Opus 5	29	37	1	16	83	0.684	0.795
Claude Sonnet 5	13	7	1	5	26	0.835	0.918
DeepSeek Flash	14	29	3	10	56	0.696	0.772
GPT-5.6 Terra	8	1	0	3	12	0.871	0.957

Note. Adjusted precision = (TP + real outside ground truth) / (TP + FP); recall is unchanged, and no adjusted F1 is reported (Section 2.4).

Enabling native reasoning did not help (Table 6). Opus moved from 0.688 to 0.684, Sonnet from 0.632 to 0.579, DeepSeek from 0.577 to 0.544, and GPT from 0.459 to 0.494; none of the four differences excludes zero. Every model became more precise with reasoning on; Opus, Sonnet, and DeepSeek became less thorough while GPT's recall rose. Latency rose 1.7 to 4.0 times, output tokens 1.6 to 4.7 times, and cost per review 1.4 to 2.5 times.

Table 6: Effect of Native Reasoning Mode (System-Prompt Enrichment, Single Agent)

Model	F1 off	F1 on	Difference	95% CI	p	Latency off → on (s)	Empty-list runs (on)	Non-JSON runs (on)
Claude Opus 5	0.688	0.684	−0.004	[−0.042, +0.030]	0.83	14.8 → 25.6	0	0
Claude Sonnet 5	0.632	0.579	−0.052	[−0.102, +0.007]	0.08	7.6 → 15.1	0	0
GPT-5.6 Terra	0.459	0.494	+0.035	[−0.014, +0.089]	0.18	4.4 → 8.7	2	0
DeepSeek Flash	0.577	0.544	−0.033	[−0.091, +0.029]	0.29	3.6 → 14.2	0	0

Note. Empty-list runs are scored as empty findings lists.

Table 7: Public Benchmark Standing at Time of Writing Versus Observed Rank

Model	Artificial Analysis Intelligence Index (rank of four)	SWE-bench Verified	Observed F1 rank
GPT-5.6 Terra	55.0 (1)	not listed	4
Claude Opus 5	50.7 (2)	96%	1
Claude Sonnet 5	38.4 (3)	85.2%	2
DeepSeek Flash	34.5 (4)	79%	3

Note. Leaderboard values as read on September 15, 2026 (Artificial Analysis, 2026; BenchLM, 2026).

3.1.4. Comparison with Hypothesis

The hypothesis that model selection influences detection is supported, and this effect is the largest in the study. The best and worst models are 0.229 F1 apart, every pairwise difference against the leader is significant, and the effect survives the audit of rejected claims. The difference is thoroughness rather than accuracy, so the choice is between finding more and paying more, and a general-purpose leaderboard was no guide to it.

3.2. Context Engineering Testing and Findings

3.2.1. Introduction

This leg asks whether the way a file is presented to the model affects what the model finds, and whether the answer is the same across models. Each of the four models reviewed the six files across the five prompt variants in Section 2.3 using a single call. The paired comparison for each model is against its own zero-shot run: the plain task instruction with no persona, examples, guidance, or reasoning steps. System-prompt

enrichment was carried forward as the fixed prompt for Sections 3.1 and 3.3 because it was the only variant to beat zero-shot in a pilot on the original, unmutated corpus. That decision was made before any run in this study.

3.2.2. Presentation of Data

Table 8 reports F1 for each model across all variants, and Table 9 reports each variant's paired difference from the same model's zero-shot run. On the two Claude models, no variant beat zero-shot, and two were worse on both: few-shot (Opus −0.033, $p = 0.03$; Sonnet −0.072, $p < 0.001$) and chain-of-thought (Opus −0.022, $p = 0.03$; Sonnet −0.059, $p = 0.002$); Sonnet's two differences meet the study's $p \leq 0.01$ threshold, and Opus's two fall short of it. On DeepSeek, no variant moved F1 by more than 0.035, and none was significant. On GPT-5.6 Terra, guidance injection raised F1 by 0.073 ($p < 0.001$), from 0.454 to 0.527; few-shot's gain of 0.043 did not reach significance.

Table 8: F1 by Model and Context Variant (Single Agent; 30 Runs per Cell)

Model	Zero-shot	Few-shot	System-prompt enrichment	Guidance injection	Chain-of-thought
Claude Opus 5	0.685	0.652	0.688	0.684	0.663
Claude Sonnet 5	0.664	0.592	0.632	0.638	0.605
DeepSeek Flash	0.566	0.570	0.577	0.565	0.531
GPT-5.6 Terra	0.454	0.497	0.459	0.527	0.446

Table 9: Paired F1 Difference Against the Same Model's Zero-Shot Run

Model	Few-shot	System-prompt enrichment	Guidance injection	Chain-of-thought
Claude Opus 5	−0.033 (p = 0.03)	+0.003 (0.81)	−0.001 (0.94)	−0.022 (0.03)
Claude Sonnet 5	−0.072 (<0.001)	−0.032 (0.25)	−0.026 (0.34)	−0.059 (0.002)
DeepSeek Flash	+0.004 (0.89)	+0.011 (0.68)	−0.001 (0.92)	−0.035 (0.12)
GPT-5.6 Terra	+0.043 (0.16)	+0.005 (0.77)	+0.073 (<0.001)	−0.008 (0.64)

Note. Difference in F1 with two-sided p in parentheses; 20,000-iteration paired bootstrap on shared file-and-repetition units.

3.2.3. Detailed Analysis

The effect of a prompt depended more on the model than on the prompt itself. Elaborating the prompt hurt the strongest models and helped the weakest. On Sonnet,

every addition to the plain instruction narrowed what the model found: 144 true positives with zero-shot, 124 with few-shot, and 130 with chain-of-thought, while its precision stayed between 0.77 and 0.84. On GPT, guidance text raised recall from 0.315 to 0.377 (82 to 98 true positives) and precision from 0.81 to 0.88. The guidance supplied checks GPT was not making on its own and that the Claude models already made.

Few-shot lowered both Claude models' recall by the same route. The few-shot examples were written for Azure and GCP so that no AWS patterns would prime the model, and the Claude models appear to have narrowed to the classes shown in the examples. The chain-of-thought instruction ends with a step that discards findings the model cannot substantiate, which is the same pruning the critic pattern performs in Section 3.3, with the same result on Sonnet; on Opus it left recall unchanged and added false positives, lowering precision from 0.672 to 0.633. Neither variant hurt GPT, which had little to prune.

The enriched persona prompt, the pilot's winner, did not beat zero-shot on any model here: Opus +0.003, Sonnet −0.032, DeepSeek +0.011, GPT +0.005, none significant. Two readings fit, and the study cannot separate them. With five repetitions per file, the intervals are wide enough for a three-point effect to change sign between two runs of the same configuration. Moreover, a persona prompt may help more when the model can recognize the corpus it is reading, which the mutation removed. Either way, carrying the enriched prompt forward changed nothing measurable: it is within noise of zero-shot on every model.

The gains, where they occurred, were small relative to the model effect. GPT's best variant, 0.527, remained below Sonnet's worst, 0.592, and 0.158 below Opus with no prompt engineering at all. The largest prompt effect in the leg, 0.073, is a third of the model spread in Table 2.

3.2.4. Comparison with Hypothesis

The hypothesis that context engineering improves detection is supported for one model and contradicted for two. Guidance injection improved GPT-5.6 Terra; nothing improved DeepSeek; few-shot and chain-of-thought reduced both Claude models, Sonnet at the

study's threshold and Opus short of it. Where the technique helped, it helped a model that was under-reporting, and the improvement did not close the gap to any stronger model.

3.3. AI Agent Orchestration Patterns Testing and Findings

3.3.1. Introduction

This leg asks whether wrapping a single review call in a multi-agent pattern improves detection, for which models, and at what cost. The four patterns are described in Section 2.3. Two details matter for reading the results. The debate pattern samples the same model twice and reconciles the two reviews with a third call, so the disagreement it resolves is sampling variance within one model rather than a difference between models; like the multi-agent debate of Du et al. (2024), it uses instances of one model, but without rounds in which the reviewers see each other's answers. The chunked pattern never splits a Terraform block, and every chunk carries a list of all resources in the file so that cross-resource reasoning remains possible. The 2,500-character chunk budget split only db-app.tf (four chunks) and ec2.tf (three); the other four files fit in one chunk and were sent whole, under a header that presents the text as an excerpt of a larger file and forbids findings on resources not shown, so on those files the pattern differs from the single call only by that header. Every pattern was run on all four models with the enriched prompt, and each was paired against that model's single-call configuration from Section 3.1.

3.3.2. Presentation of Data

Table 10 gives detection and cost for all sixteen model-and-pattern combinations, and Table 11 gives the paired difference from single. For the Claude models, no pattern helped, and two lowered F1: chunked on Opus (−0.078, $p < 0.001$), at the study's threshold, and critic on Sonnet (−0.073, $p = 0.02$), short of it. Debate was indistinguishable from single on both. For DeepSeek, debate added 0.048 ($p = 0.013$); critic and chunked did not move it. For GPT-5.6 Terra, debate added 0.064 ($p = 0.02$) and chunked added 0.075 ($p = 0.006$); critic did not. Cost multiples were the same across models: 1.2 to 1.5 times for chunked, 2.1 to 2.3 for critic, and 3.3 to 3.6 for debate.

Table 10: Detection Performance and Cost by Orchestration Pattern (System-Prompt Enrichment)

Model	Pattern	Precision	Recall	F1	F1 95% CI	Calls	Cost per review (USD)	Cost multiple
Claude Opus 5	Single	0.684	0.692	0.688	[0.636, 0.735]	1.0	0.047	1.0
Claude Opus 5	Critic	0.687	0.642	0.664	[0.606, 0.716]	2.0	0.099	2.1
Claude Opus 5	Debate	0.672	0.692	0.682	[0.625, 0.731]	3.0	0.155	3.3
Claude Opus 5	Chunked	0.629	0.592	0.610	[0.559, 0.661]	1.8	0.056	1.2
Claude Sonnet 5	Single	0.835	0.508	0.632	[0.576, 0.682]	1.0	0.014	1.0
Claude Sonnet 5	Critic	0.850	0.415	0.558	[0.485, 0.629]	2.0	0.030	2.1
Claude Sonnet 5	Debate	0.845	0.523	0.646	[0.593, 0.695]	3.0	0.047	3.3
Claude Sonnet 5	Chunked	0.764	0.512	0.613	[0.559, 0.670]	1.8	0.020	1.4
DeepSeek Flash	Single	0.696	0.492	0.577	[0.517, 0.633]	1.0	0.0004	1.0
DeepSeek Flash	Critic	0.784	0.419	0.546	[0.473, 0.617]	2.0	0.0010	2.2
DeepSeek Flash	Debate	0.745	0.538	0.625	[0.555, 0.687]	3.0	0.0016	3.5
DeepSeek Flash	Chunked	0.683	0.488	0.570	[0.513, 0.622]	1.8	0.0006	1.4
GPT-5.6 Terra	Single	0.871	0.312	0.459	[0.405, 0.512]	1.0	0.006	1.0
GPT-5.6 Terra	Critic	0.870	0.335	0.483	[0.426, 0.531]	2.0	0.014	2.3
GPT-5.6 Terra	Debate	0.874	0.373	0.523	[0.463, 0.574]	3.0	0.022	3.6
GPT-5.6 Terra	Chunked	0.817	0.396	0.534	[0.472, 0.585]	1.8	0.009	1.5

Note. Cost per review at August 2026 list prices; fabricated resource addresses (claims naming a resource not in the file) rose with chunking on every model: Opus 0 to 5, Sonnet 0 to 5, DeepSeek 5 to 9, GPT 3 to 5.

Table 11: Paired F1 Difference Against the Same Model's Single-Agent Configuration

Model	Critic	Debate	Chunked
Claude Opus 5	−0.024 (p = 0.12)	−0.007 (0.58)	−0.078 (<0.001)
Claude Sonnet 5	−0.073 (0.02)	+0.015 (0.42)	−0.019 (0.34)
DeepSeek Flash	−0.030 (0.34)	+0.048 (0.013)	−0.007 (0.72)
GPT-5.6 Terra	+0.024 (0.40)	+0.064 (0.02)	+0.075 (0.006)

3.3.3. Detailed Analysis

The pattern is the one Section 3.2 found. Orchestration hurt the strong models or did nothing for them, and helped the weak ones; every gain was recall. GPT's true positives rose from 81 to 97 with debate and to 103 with chunked; DeepSeek's rose from 128 to 140 with debate; precision held or improved except for GPT's chunked arm, where it fell from 0.871 to 0.817. On Opus, no pattern added a true positive over the single call (debate 180, single 180), and on Sonnet, debate added four.

The critic pattern shortened the list on every model and ended with fewer true positives than the single call on three of the four. Sonnet's critique pass cut its findings from 164 to 127 and shrank the list in 24 of 30 runs; true positives fell from 132 to 108 while precision rose only from 0.835 to 0.850. Opus went from 260 to 243 findings (the list shrank in 18 of 30 runs), DeepSeek from 191 to 139 (17 of 30 runs), and GPT from 109 to 100 (11 of 30 runs). The pattern is consistent with a second opinion from the same model acting as a filter that does not distinguish correct findings from incorrect ones; only the counts per step were kept, so which findings a critique removed cannot be shown. GPT's critic configuration was the only one to end with more true positives than its single call (87 against 81), and it had the fewest findings to remove.

Debate paid off only where a single pass under-reported. The reconciled list is consistent with the union of the two reviews after light pruning: Opus's 263 and 257 became 268, GPT's 113 and 111 became 111, and DeepSeek's 179 and 195 became 188. A model that finds a third of the flaws on each pass would find different ones on the second pass, so the union is larger, while a model already finding two-thirds draws two overlapping samples; the counts fit that reading, though the overlap itself was not measured.

Chunking cut both ways, and the file-level counts show where. On Opus, it lost 26 true positives (180 to 154): 24 of them on the four files that were never split, and 22 on s3.tf alone, where the public-access-block findings on four buckets fell from five runs in five to none or one; on the two files that were split, Opus lost 2 true positives and added 10 false positives. What cost Opus was the chunk header on a whole file, most plausibly its instruction not to report against resources it cannot see, applied to a finding about a resource the file lacks. On GPT, chunking gained 22 true positives (81 to 103), 23 of them on the two split files (ec2.tf 20 to 39) and one fewer on the unsplit ones, so for GPT the splitting itself helped. Chunking also produced 5 fabricated resource addresses across Opus's 245 claims, where the single call produced none across 263, and fabrication rose with chunking on every model.

No pattern was cost-effective. The cost multiple exceeded the gain everywhere, so F1 per dollar fell for every pattern on every model, including every pattern that raised F1. GPT's debate arm raised F1 by 14% for 3.6 times the cost. Among the Claude configurations in

Table 10, single-call Sonnet returned the most F1 per dollar, 0.632 at $0.014 per review against Opus's 0.688 at $0.047; across all four models, single-call DeepSeek did.

The split between strong and weak models follows from the corpus. The planted flaws are local, single-attribute misconfigurations. A model that catches most of them on the first pass leaves little for a second pass to find, so a second pass has little left to add. A model that catches a third of them leaves plenty. Orchestration multiplied recall for the models that had recall to gain. Where the corpus does contain relational flaws, in iam.tf, every model struggled, and no pattern closed the gap.

3.3.4. Comparison with Hypothesis

The hypothesis that orchestration improves detection is supported for GPT-5.6 Terra by chunking ($p = 0.006$), suggested but not established at the study's threshold for GPT debate and DeepSeek debate ($p = 0.02$ and 0.013), and contradicted for the two strongest models; it fails on cost for all four. No pattern lifted a weaker model to the next model's single-call score: GPT's best pattern, 0.534, sits below Sonnet's worst, 0.558, and 0.154 below Opus alone. Every pattern cost more per point of F1 than the single call it wrapped.

3.4. Integrity Checks

3.4.1. Contamination Control

TerraGoat has been public and unchanged since April 2023; every model tested was trained after that date, and the original files carry tags naming the repository. A model could therefore score well by recognizing the corpus rather than reading it. The study ran on a mutated copy in which resource names and references, identifier strings, HCL comments, provenance tags, and block order were rewritten (Section 2.1). The rewrite left recognizable material behind: the shell comments inside two user_data scripts, including a link to the AWS tutorial the code was taken from, and the structure and resource set of every file. It also introduced two defects that the models could see: the six reviewed files reference local.env_prefix while consts.tf still defines resource_prefix, so the mutated corpus does not validate as Terraform, and the rule that renamed ebs changed

the AMI name filter in db-app.tf to a value that matches no image. Neither touches a ground-truth attribute, and Checkov's result was unaffected. Checkov's 95 failed checks were identical in the original and mutated files once the resource names were mapped back, and a fresh scan of the original matched the August baseline with no drift. The top of the ranking from the pilot on the original corpus, Opus, then Sonnet, was reproduced on the mutated corpus; the two weakest models, within a point of each other in the pilot, were separated here. The study did not run the original corpus alongside, so it reduces the opportunity for memorization rather than measuring it; how much a public corpus inflates scores is not known from this data.

A third defect was caught during the runs, and the models found it. The first mutated s3.tf carried an invalid ACL value produced by a string-replacement rule that matched inside log-delivery-write, and two models flagged it as a finding. The rule was corrected, the corpus regenerated and re-validated with the same 95 checks matching, and every run on the two affected files was repeated across all configurations then existing. The four unchanged files were byte-identical before and after, and their runs stand. The superseded runs remain in the repository's history.

3.4.2. Oracle Completeness

The ground truth is what Checkov can see, which is why Section 2.4 treats precision as a floor. To measure how far below the true value the floor sits, every claim the scorer rejected in the model comparison, 177 across the four models, was read against the mutated Terraform and placed in one of four groups: a real vulnerability the ground truth does not contain; a true observation the study's criterion excludes; a model error; or a duplicate of a finding the same run had already been credited for. An AI model produced the first pass of that classification, and the author then reviewed every row against the code; all 177 first-pass verdicts were accepted.

Table 5 gives the result. Model errors were rare: 5 of 177, none for GPT, and one for Sonnet. The largest group, 74, were findings that the criterion excludes on purpose, almost all of them versioning, access logging, and force_destroy on S3 buckets, which Opus and DeepSeek report on nearly every bucket in every run. The next largest, 64,

were real weaknesses that the ground truth could not see. Adjusted precision credits that group and leaves the rest as scored. The credit is not exact: 19 of the 64 attribute the IAM access-key exposure to the Terraform output, and that output references encrypted_secret, which is empty when no PGP key is set, so the output exposes nothing while the plaintext secret in state does. What remains as false positives after adjustment is the 74 observations the criterion excludes by design, the 34 duplicates, and the 5 errors.

4. Recommendations and Implications for Future Research

4.1. Recommendations for LLM Model Selection

4.1.1. Key Findings

Model choice was the largest effect in the study: 0.229 F1 between the best and worst of four models, every difference against the leader was significant, and the ranking remained unchanged after the audit of rejected claims. The difference was recall. Precision ran from 0.68 to 0.87 as scored and from 0.77 to 0.96 under the adjustment bounds; recall ran from 0.69 to 0.31. A general-purpose leaderboard predicted the wrong order, and a coding benchmark predicted the right one for the models it listed. Native reasoning helped no model.

4.1.2. Implications

For security review, recall is the number to select on. A false positive costs a reviewer minutes; a false negative ships a misconfiguration to production, which is the failure the review exists to prevent (Verdet et al., 2025). Teams should pick the most thorough model their budget allows and test the candidates on their own code before choosing, because leaderboard rank did not transfer to this task. Within a model family, the mid-tier model was the value choice: Sonnet 5 delivered 92% of Opus 5's F1 at 31% of the cost. Reasoning mode should be left off for this task; it raised latency 1.7 to 4.0 times and output tokens 1.6 to 4.7 times, and made every model more precise and three of the four less thorough.

4.1.3. Recommendations for Future Research

Running the original and mutated corpora side by side would measure how much of a model's score on a public benchmark corpus is recall of the corpus itself; this study only defended against it. A corpus of production IaC, with ground truth from an audit rather than a scanner, would test whether the ranking holds on the subtler flaws that reach real environments. A locally hosted model would complete the deployment spectrum; the one attempted in the pilot returned no parseable output.

4.2. Recommendations for Context Engineering

4.2.1. Key Findings

No prompt variant improved either of the Claude models, and two, few-shot and chain-of-thought, reduced them, Sonnet at the study's threshold and Opus short of it. Guidance injection improved GPT-5.6 Terra by 0.073 F1; nothing moved DeepSeek. The enriched persona prompt that won a pilot on the original corpus was within noise of zero-shot across all models on the mutated corpus. No variant on any model closed the gap to the next model up.

4.2.2. Implications

Choose the prompt after the model, not before. For a frontier model, a clear task instruction is enough, and elaboration tends to narrow what the model reports. For the weakest model here, supplying the checklist as guidance text recovered some recall at a small token cost. Claims that a prompting technique improves detection should be tested on the model in use, with repetitions, before being adopted; the effects here were smaller than the confidence intervals of single configurations. The exact prompts are published with the harness so the comparison can be repeated.

4.2.3. Recommendations for Future Research

Real retrieval, selecting guidance per file from a control catalog, is the natural next step from static injection, and the GPT result suggests where it would pay. Composed

variants, such as a persona prompt with guidance, were not tested. Because the Claude failure mode was under-reporting, prompts that ask the model to be exhaustive rather than conservative are the untested direction most likely to move recall.

4.3. Recommendations for AI Agent Orchestration Patterns

4.3.1. Key Findings

Orchestration raised detection only for the two low-recall models (GPT chunked +0.075 at the study's threshold; GPT debate +0.064 and DeepSeek debate +0.048 short of it) and lowered it for the Claude models (Opus chunked −0.078 at the threshold; Sonnet critic −0.073 short of it). Cost multiples ranged from 1.2 to 3.6 across all models, so F1 per dollar fell everywhere. The critic pattern resulted in fewer true positives across three of the four models. Chunking fabricated resource addresses on every model.

4.3.2. Implications

The orchestration budget buys more as a model upgrade. GPT-5.6 Terra, with debate at 3.6 times the cost, reached 0.523; Sonnet 5, with a single call, reached 0.632 for less. If a team is fixed to a weaker model, a second independent sample reconciled by a third call is the pattern that helps, and a critic pass is the one to avoid: on three of the four models, a same-model critic ended with fewer correct findings than the single call. On a strong model, the chunk-mode header cost more than the splitting did (Section 3.3.3), so a whole file should be reviewed as a whole file, without excerpt framing; splitting itself helped only GPT. Every reported resource address should be checked against the file, regardless of the model, since chunking raised fabrication rates everywhere.

4.3.3. Recommendations for Future Research

The planted flaws here are local, which is why a second pass added nothing to a strong model. A corpus of relational flaws, such as privilege-escalation chains spanning roles, policies, and compute in CloudGoat (Rhino Security Labs, 2018), is the test that could reward orchestration with strong models. Cross-model debate, a cheap reviewer reconciled by a strong one, was not tested and may combine the recall gain seen here

with the strong model's judgment. A tool-calling agent that runs an independent scanner and interprets its output was designed for this study but was not run.

5. Conclusion

This study asked whether four implementation choices change how well an LLM finds security flaws in Terraform: the model, the prompt, the agent pattern, and native reasoning. Four models reviewed six files of a mutated TerraGoat corpus across five prompt variants and four orchestration patterns, each run five times, yielding 1,080 scored runs, with every figure recomputable from the published raw results.

Model choice mattered most, and what it changed was recall. Claude Opus 5 found 69% of the planted checks; GPT-5.6 Terra found 31%; GPT was the more precise of the two, and the difference between them was recall. Prompt design and multi-agent patterns helped only the models that were under-reporting, by five to eight F1 points, lowered the models that were not, and cost up to 3.6 times as much; none brought a weaker model up to the next model's single-call score. Reasoning mode helped no model. An audit of every rejected claim in the model comparison found that a third were real weaknesses the scanner-derived ground truth could not see, and another two-fifths were true observations outside the study's criterion, which raises every model's precision bound and leaves the ranking alone.

For a cloud security team, the recommendations follow directly. Choose a model for recall and test it on your own code, since the leaderboards did not predict this task. With a frontier model, keep the prompt plain and skip the agents; the budget is better spent on the model. With a constrained model, guidance text or a second sample each recovered some recall on its own, at a cost; the combination was not tested. For the field, the open questions are the ones this design could not reach: how much a public corpus inflates scores through memorization, whether relational flaws reward orchestration where local ones did not, and whether the ranking holds on production code.

Use of AI Tools

The evaluation harness, the corpus mutation script, and the scripts that score the runs and verify the reported figures were developed with the assistance of an AI coding model (Claude, Anthropic), which also executed the experiment runs. The first-pass classification of rejected claims in Section 3.4.2 was produced by the same model and reviewed row by row by the author, who accepted all 177 verdicts without change. Sections 3 through 5 and the abstract were drafted with AI assistance from the author's verified results and revised and validated by the author. Two independent audits of the draft and its evidence were run with ChatGPT (OpenAI), and each finding was checked against the code and data before any change was made; Grammarly was used for copyediting. All findings, interpretations, and language are the author's responsibility.

References

Amazon Web Services. (n.d.). *AWS Well-Architected Framework: Security pillar*. https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/welcome.html

Anthropic. (2026). *Models overview*. https://platform.claude.com/docs/en/about-claude/models/overview

Artificial Analysis. (2026). *Artificial Analysis Intelligence Index* [Leaderboard]. Retrieved September 15, 2026, from https://artificialanalysis.ai/

BenchLM. (2026). *SWE-bench Verified leaderboard*. Retrieved September 15, 2026, from https://benchlm.ai/benchmarks/swe-bench-verified

Bridgecrew. (2020a). *Checkov: Policy-as-code for everyone* [Computer software]. GitHub. https://github.com/bridgecrewio/checkov

Bridgecrew. (2020b). *TerraGoat: Vulnerable by design Terraform repository* [Computer software]. GitHub. https://github.com/bridgecrewio/terragoat

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., … Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems, 33*, 1877–1901.

Center for Internet Security. (2026). *CIS Amazon Web Services Foundations Benchmark* (Version 7.0.0). https://www.cisecurity.org/benchmark/amazon_web_services

Crosley, B. (2026, January). *Infrastructure automation with AI: Using LLMs to generate IaC scripts*. Introl. https://introl.com/blog/infrastructure-automation-ai-using-llms-generate-iac-scripts

DeepSeek. (2026). *DeepSeek-V4-Flash-0731* [Large language model]. Hugging Face. https://huggingface.co/deepseek-ai/DeepSeek-V4-Flash-0731

Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., & Mordatch, I. (2024). Improving factuality and reasoning in language models through multiagent debate. *Proceedings of the 41st International Conference on Machine Learning, PMLR 235*, 11733–11763.

Firouzi, E., Bhatt, S., & Ghafari, M. (2026). *Can developers rely on LLMs for secure IaC development?* arXiv. https://arxiv.org/abs/2602.03648

He, H. (2025, September). *Defeating nondeterminism in LLM inference*. Thinking Machines Lab. https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, *33*, 9459–9474.

Open Worldwide Application Security Project. (n.d.). *Infrastructure as Code security cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Infrastructure_as_Code_Security_Cheat_Sheet.html

Open Worldwide Application Security Project. (2024). *OWASP Top 10 for LLM applications 2025*. https://owasp.org/www-project-top-10-for-large-language-model-applications/

OpenAI. (2026). *Models*. https://developers.openai.com/api/docs/models

Rhino Security Labs. (2018). *CloudGoat: "Vulnerable by design" AWS deployment tool* [Computer software]. GitHub. https://github.com/RhinoSecurityLabs/cloudgoat

Verdet, A., Hamdaqa, M., Da Silva, L., & Khomh, F. (2025). Assessing the adoption of security policies by developers in Terraform across different cloud providers. *Empirical Software Engineering*, *30*(3), Article 74. https://doi.org/10.1007/s10664-024-10610-0

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems, 35*, 24824–24837.